SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION WITH OPENCV
COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry.* (BTL 6)

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a **face recognition system** on low-power edge devices (e.g., Raspberry Pi). The system must:

A. Process live video streams in real time (<30 FPS latency)
B. Work under varying lighting conditions
C. Use limited memory and CPU resources
D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System

A security firm needs a **video surveillance system** that:

A. Detects motion and unusual activities
B. Generates alerts in real time
C. Works with standard CCTV video feeds
D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents

A digitization company needs an **OCR system** that:

A. Extracts text from low-resolution, noisy scanned images
B. Handles varying fonts and illumination
C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

4. Facial Expression Recognition for Mobile Application

A mobile app must detect **facial expressions (happy, sad, neutral)** with the following constraints:

A. Limited processing power (mobile CPU)
B. Real-time performance
C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

5. Smart Traffic Monitoring System

A city authority needs a system to:

A. Detect vehicles from live video streams
B. Track movement across frames
C. Count vehicles and display statistics
D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

6. Real-Time Object Detection on Mobile Device

A mobile application must:

A. Detect objects using the camera
B. Run in real time
C. Use limited CPU and memory

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how your design ensures efficiency under constraints.

7. Smart Attendance System using Face Recognition

A system must:

A. Capture faces from a webcam
B. Recognize registered users
C. Mark attendance automatically

Design a complete pipeline using OpenCV. Justify each stage and how the system avoids false recognition.

8. Motion Detection for Home Security

A home security system must:

A. Detect motion from a camera feed
B. Trigger alerts only for significant movement
C. Reduce false alarms

Design the pipeline and justify how your approach handles noise and background variations.

9. Document Scanner Application

A mobile app must:

A. Capture document images
B. Enhance readability
C. Output clean scanned images

Design the OpenCV pipeline and justify preprocessing steps used for improving document quality.

10. Vehicle Detection in Parking System

A parking system must:

A. Detect cars entering/exiting
B. Track movement
C. Update available slots

Design the pipeline and justify how your approach handles multiple vehicles and avoids counting errors.

11. Gesture Recognition System

A system must:

A. Capture hand gestures from a webcam
B. Identify simple gestures
C. Respond in real time

Design an OpenCV pipeline and justify feature extraction and classification methods.

12. Video Frame Quality Enhancement

A system must:

A. Improve video quality in real time
B. Reduce noise and enhance contrast

Design the pipeline and justify enhancement techniques used.

13. Multi-Face Tracking System

A system must:

A. Detect multiple faces
B. Track them across frames
C. Maintain identity

Design the pipeline and justify tracking strategy.

14. OCR for License Plate Recognition

A system must:

A. Detect license plates
B. Extract text
C. Work under varying lighting

Design the OpenCV pipeline and justify preprocessing and segmentation methods.

15. Real-Time Edge Detection System

A system must:

A. Capture video
B. Detect edges in real time
C. Display results efficiently

Design the pipeline and justify optimization strategies.

Code of Conduct:

I _Bhavya PS(192524017) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Application of Concepts	**15%**	**Effectively applies appropriate concepts, methods, or tools**	**Applies concepts with minor errors**	**Limited application; requires guidance**	**Unable to apply relevant concepts**
Justification	**10%**	**Provides strong justification for decisions with logical reasoning**	**Provides justification with some clarity**	**Weak or unclear justification**	**No justification provided**

1. OCR System for Low-Quality Documents

Objective

To design an OpenCV-based OCR preprocessing pipeline that improves text recognition from low-resolution, noisy scanned documents with different fonts and illumination conditions.

Proposed Pipeline

```
Input Image → Grayscale → Noise Removal → Contrast Enhancement →
  Thresholding → Morphological Processing → Segmentation → OCR
```

Procedure

1. Image Acquisition

- Read the scanned document using cv2.imread().
- Resize the image using cv2.resize() if the text is too small.
- Upscaling helps OCR identify characters more clearly.

2. Grayscale Conversion

- Convert the image into grayscale using cv2.cvtColor(image, cv2.COLOR_BGR2GRAY).
- This reduces the image from three colour channels to one and decreases processing time.

3. Noise Removal

- Apply cv2.GaussianBlur() or cv2.medianBlur().
- Median filtering is particularly useful for removing salt-and-pepper noise from scanned documents.
- This prevents noise from being incorrectly identified as characters.

4. Contrast Enhancement

- Use CLAHE (Contrast Limited Adaptive Histogram Equalization).
- It improves local contrast in regions where illumination is uneven.
- This helps both dark and faint characters become more visible.

5. Image Thresholding

- Use cv2.threshold() or cv2.adaptiveThreshold().
- Adaptive thresholding is preferred when illumination varies across the document.
- It converts the image into a suitable binary form containing foreground text and background.

6. Morphological Processing

- Use cv2.morphologyEx() with opening and closing.

- Opening removes small unwanted noise.
- Closing connects broken parts of characters.
- This improves the shape and continuity of text.

7. Text Segmentation

- Use contours with cv2.findContours() to identify text regions.
- Individual lines or regions can be extracted using bounding boxes.
- Segmentation allows OCR to process smaller and cleaner text regions.

8. OCR Recognition

- The processed image is given to an OCR engine such as Tesseract.
- Since OpenCV has already removed noise and improved contrast, the OCR engine receives a cleaner image.

Justification

Constraint	OpenCV Solution	Benefit
Low-resolution images	cv2.resize()	Makes small characters easier to recognize
Noise	medianBlur() / GaussianBlur()	Removes unwanted pixels
Uneven illumination	CLAHE	Improves local contrast
Different backgrounds	adaptiveThreshold()	Produces better text-background separation
Broken characters	Morphological closing	Connects character components
Batch processing	Automated OpenCV pipeline	Same processing can be applied to many images

Conclusion

The proposed pipeline improves OCR accuracy by enhancing low-quality images before recognition. Adaptive thresholding and CLAHE handle illumination variations, while filtering and morphological operations remove noise and repair characters. The pipeline can also be applied automatically to large batches of documents, making it suitable for digitization systems.

2. Facial Expression Recognition for Mobile Application

Objective

To develop an efficient OpenCV-based facial expression recognition system that identifies happy, sad, and neutral expressions while maintaining real-time performance on a mobile CPU.

Proposed Pipeline

```
Camera → Frame Resizing → Face Detection → Face ROI Extraction →
Grayscale → Normalization → Expression Analysis → Display Result
```

Procedure

1. Capture Video

- Capture frames from the mobile camera.
- OpenCV can use cv2.VideoCapture() for camera input.

2. Frame Resizing

- Resize frames using cv2.resize().
- A smaller frame, such as 320 × 240, reduces CPU processing requirements.
- This helps achieve real-time performance.

3. Face Detection

- Use OpenCV's Haar Cascade classifier with cv2.CascadeClassifier().
- Detect faces using detectMultiScale().

4. Region of Interest Extraction

- Once a face is detected, extract only the face region.
- Processing only the face instead of the complete frame reduces computation.

5. Preprocessing

- Convert the face region to grayscale using cv2.cvtColor().
- Resize it to a fixed size.
- Apply histogram equalization or CLAHE to reduce the effect of lighting changes.

6. Expression Feature Extraction

- Important facial regions such as the eyes, eyebrows and mouth can be analyzed.
- Features can be extracted using contours, edges, or a lightweight trained classifier.

7. Expression Classification

- A lightweight machine-learning classifier can classify the extracted facial features into Happy, Sad, and Neutral.
- A small model is preferred for mobile CPU processing.

8. Temporal Smoothing

- Instead of changing the expression for every individual frame, use predictions from several consecutive frames.
- If most recent frames indicate Happy, display Happy.
- This prevents sudden changes caused by slight head movement or temporary detection errors.

9. Display Result

- Draw the detected face using cv2.rectangle().
- Display the expression using cv2.putText().

Justification

Constraint	Technique	Benefit
Limited mobile CPU	Resize frames	Reduces computation
Real-time operation	Process only face ROI	Faster processing
Lighting changes	CLAHE / histogram equalization	Improves image consistency
Slight head movement	Face detection + temporal smoothing	Reduces unstable predictions
Multiple expressions	Lightweight classifier	Provides required classification
Low latency	Process selected frames/ROI	Improves frame rate

Conclusion

The system balances accuracy and efficiency by reducing the input resolution, detecting only the required face region, and using lightweight preprocessing and classification. CLAHE improves robustness against illumination changes, while temporal smoothing makes predictions more stable during slight head movements. Therefore, the system is suitable for real-time mobile applications.

3. Smart Traffic Monitoring System

Objective

To design an OpenCV-based traffic monitoring system that detects vehicles from live video, tracks their movement, counts vehicles, and displays traffic statistics with minimum delay.

Proposed Architecture

```
Live Camera → Frame Capture → Preprocessing → Vehicle Detection →
    Vehicle Tracking → Counting → Statistics → Visualization
```

Procedure

1. Live Video Acquisition

- Capture traffic video using cv2.VideoCapture().
- Frames are continuously read from the traffic camera.

2. Frame Preprocessing

- Resize the frame using cv2.resize() to reduce processing time.
- Convert frames to grayscale when required.
- Noise reduction can be performed using Gaussian filtering.

3. Vehicle Detection

- Vehicles can be detected using background subtraction such as cv2.createBackgroundSubtractorMOG2().
- Moving objects are separated from the static background.

4. Noise Removal

- Apply morphological operations using cv2.morphologyEx().
- Small noise regions are removed and broken vehicle regions are connected.

5. Contour Detection

- Use cv2.findContours().
- Contours corresponding to vehicles are identified.
- cv2.boundingRect() provides the position and size of each detected vehicle.

6. Vehicle Tracking

- Each detected vehicle is assigned an ID.
- The position of the vehicle is compared across consecutive frames.
- A lightweight tracker such as centroid-based tracking can be used.
- The centroid of a vehicle is calculated from its bounding box.

7. Vehicle Counting

- Define a virtual counting line in the road.
- When the centroid of a tracked vehicle crosses the line, the vehicle count is increased.
- Each vehicle ID is counted only once to avoid duplicate counting.

8. Statistics Generation

- The system maintains statistics such as total vehicles, vehicles entering, vehicles leaving, current traffic density, and vehicle movement rate.

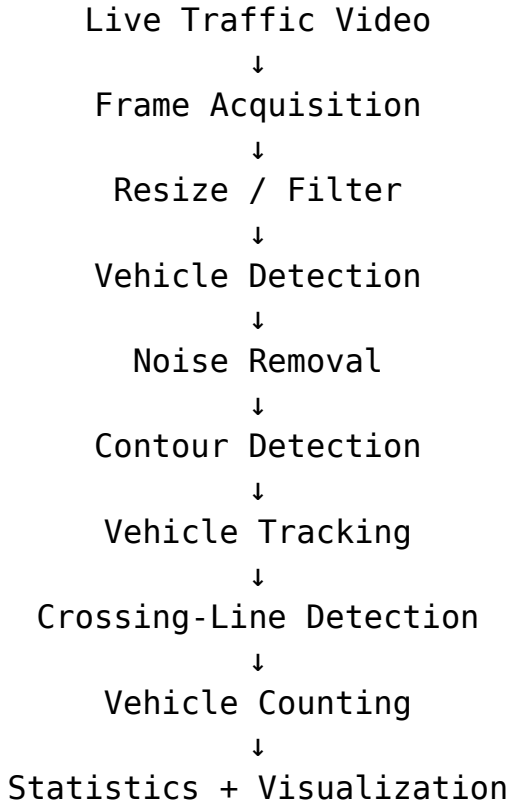
9. Visualization

- Draw bounding boxes using cv2.rectangle().
- Draw vehicle IDs using cv2.putText().
- Draw the counting line using cv2.line().
- Display the output using cv2.imshow().

10. Continuous Processing

- Frames are processed continuously until the camera is stopped.
- cv2.waitKey() is used to control frame processing and exit the application.

Simplified Flow



Justification

Requirement	OpenCV Technique	Benefit
Live video	VideoCapture()	Continuously receives camera frames

Requirement	OpenCV Technique	Benefit
Vehicle detection	MOG2 / contour detection	Detects moving vehicles
Tracking	Centroid-based tracking	Maintains vehicle identity between frames
Counting	Virtual line + centroid	Avoids repeated counting
Visualization	rectangle(), line(), putText()	Clearly displays results
Minimal delay	Frame resizing + ROI processing	Reduces computation
Continuous operation	Frame-by-frame processing	Supports live monitoring

Accuracy and Real-Time Performance

The system can maintain real-time performance by reducing frame resolution, processing only the road/vehicle region of interest, and using lightweight detection and tracking techniques. Accuracy can be improved by filtering very small contours, using appropriate area thresholds, and maintaining a unique ID for each vehicle. Temporal tracking also prevents the same vehicle from being counted multiple times.

Conclusion

The proposed OpenCV-based traffic monitoring system provides a complete solution for vehicle detection, tracking, counting, and visualization. Background subtraction detects moving vehicles, contours identify their boundaries, tracking maintains their movement across frames, and virtual-line crossing provides vehicle counts. Optimized frame processing and region-based analysis allow the system to operate continuously with low delay.